AWS KMS, AWS LAMBDA & API GATEWAY

AWS KMS

Introduction of AWS KMS: AWS Key Management Service (AWS KMS) is a fully managed encryption service provided by AWS that helps users create, manage, and control cryptographic keys used to encrypt data. KMS integrates with many AWS services such as Amazon S3, Amazon EBS, Amazon RDS, AWS Lambda, and Amazon Secrets Manager, enabling secure data protection without managing encryption infrastructure.

2. Objective

- Learn AWS KMS concepts.
- Create a Customer Managed Key (CMK).
- Encrypt and decrypt data.
- Configure an S3 bucket with SSE-KMS encryption.
- Verify KMS activity using CloudTrail.

3. Architecture

User

|

|

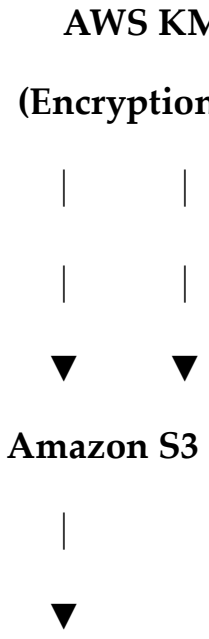
AWS Management Console

|

Create Customer Managed Key

|

▼



4. Prerequisites

AWS Account

IAM User

Internet Connection

AWS Region:

N. Virginia (us-east-1)

Create a Customer Managed Key

Step 1: Login to the AWS Console.

Search for **KMS**.

Open **Key Management Service**.

Click **Customer managed keys**.

Click **Create key**.

Step 2: Configure the key:

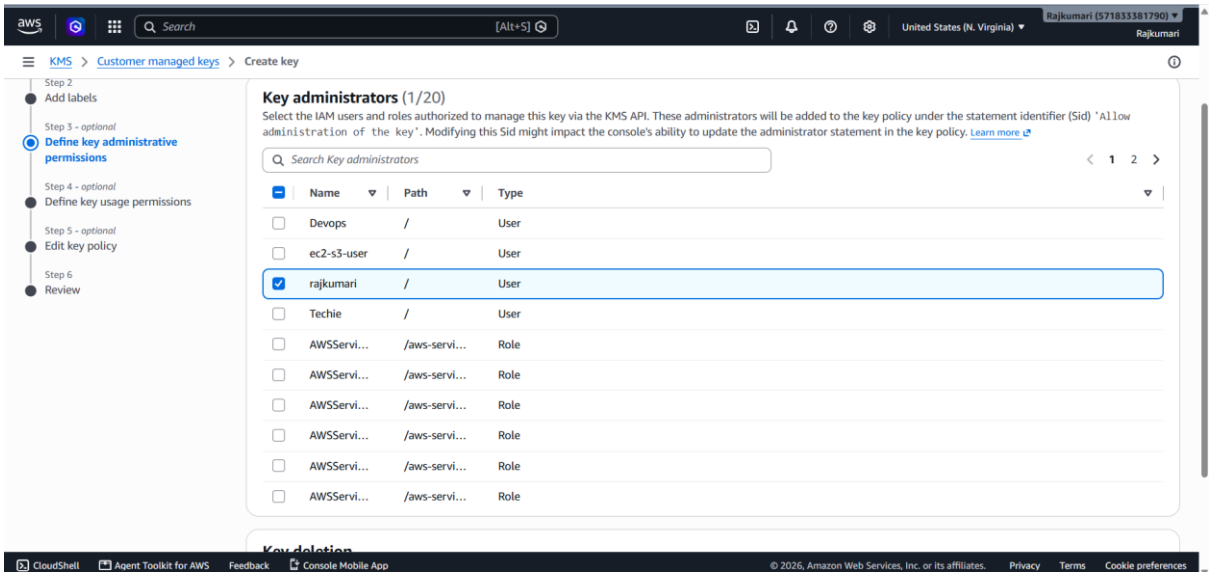
Setting	Value
Key type	Symmetric
Key usage	Encrypt and Decrypt
Origin	AWS KMS

Click **Next**.

Step 3: Choose administrators.

Select your IAM user.

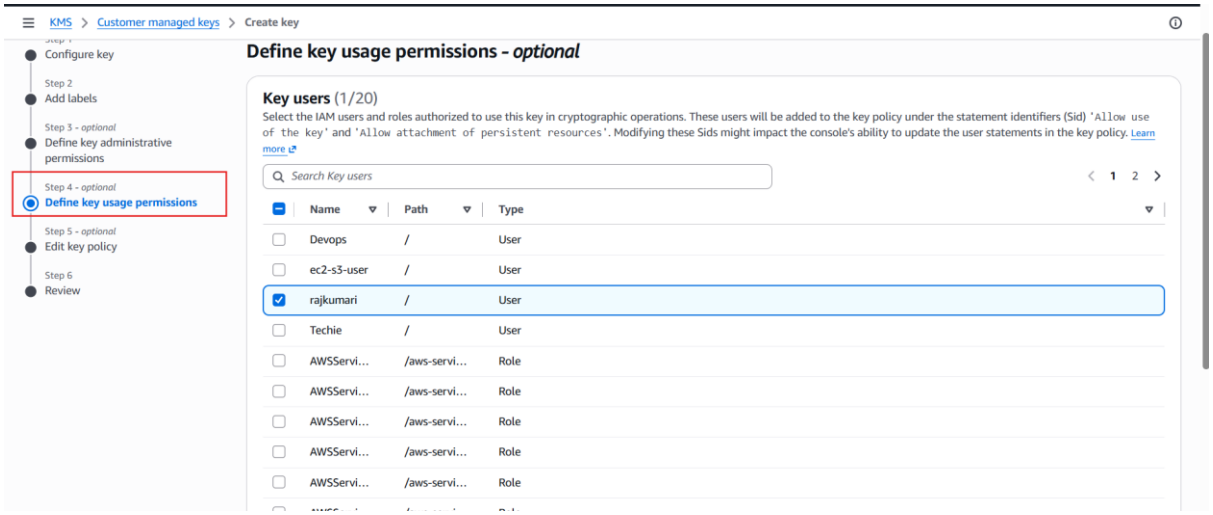
Click **Next**.



Step4: Choose key users.

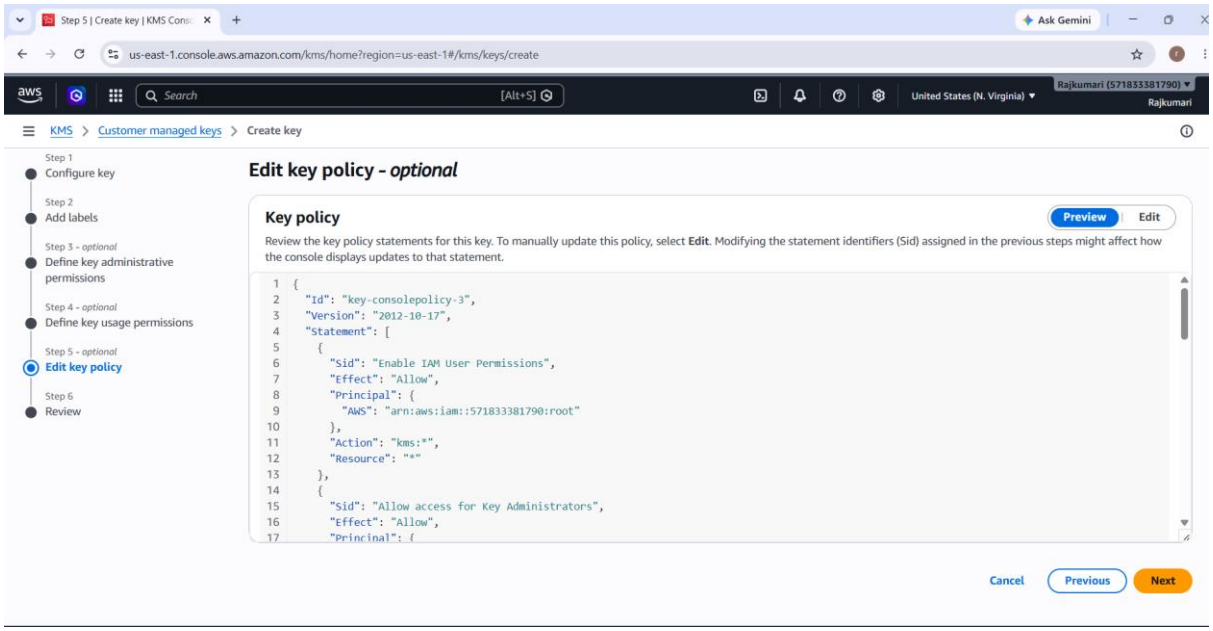
Select your IAM user.

Click **Next**.

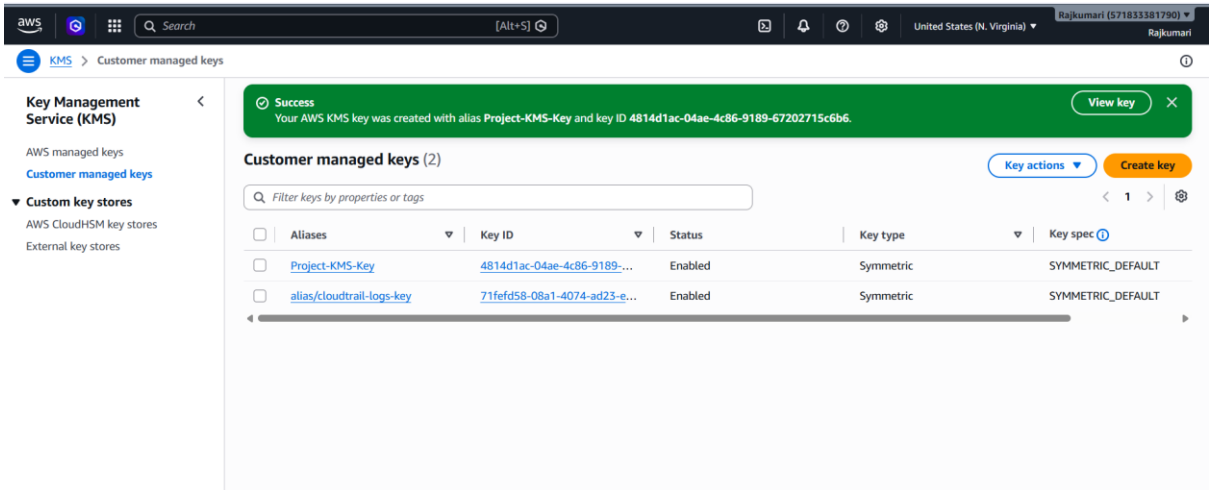


Step5: Review everything.

Click **Finish**.



Customer Managed Key Created Successfully.



Task 2: Encrypt an Amazon S3 Bucket Using AWS KMS

Objective: Configure an Amazon S3 bucket to use **Server-Side Encryption with AWS KMS (SSE-KMS)** and verify that uploaded objects are encrypted using a Customer Managed Key.

Amazon S3 is an object storage service used to securely store and retrieve files. It integrates with AWS KMS to provide server-side encryption using customer-managed encryption keys.

Step 1: Open Amazon S3

1. Log in to the AWS Management Console.
2. Search for **S3**.
3. Open the **Amazon S3** service.

Step 2: Create a New Bucket

1. Click **Create bucket**.
2. Enter a globally unique bucket name.

Example: rajkumari-kms-demo-2026

3. Select the AWS Region.

Example: US East (N. Virginia) – us-east-1

4. Leave the remaining settings as default unless required.

Step 3: Configure Default Encryption

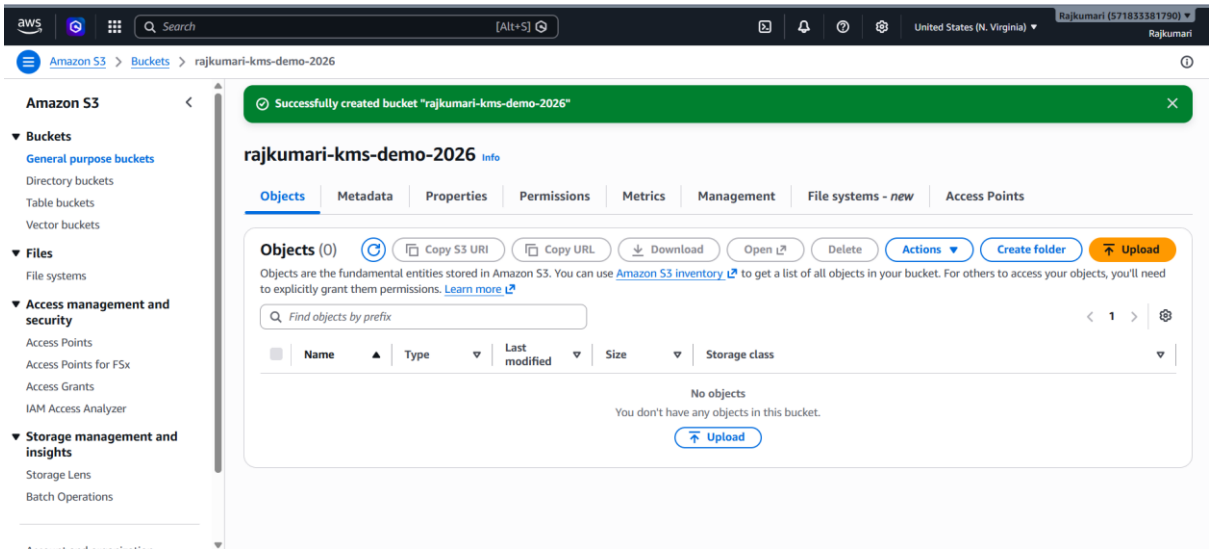
Scroll to the **Default encryption** section.

1. Select **Server-side encryption**.
2. Choose **Server-side encryption with AWS Key Management Service keys (SSE-KMS)**.
3. Under **AWS KMS key**, select **Choose from your AWS KMS keys**.
4. Select the Customer Managed Key you created earlier (for example, **Project-KMS-Key**).

SSE-KMS encrypts every object uploaded to the bucket using the selected Customer Managed Key, ensuring centralized key management and secure data protection.

Step 4: Create the Bucket

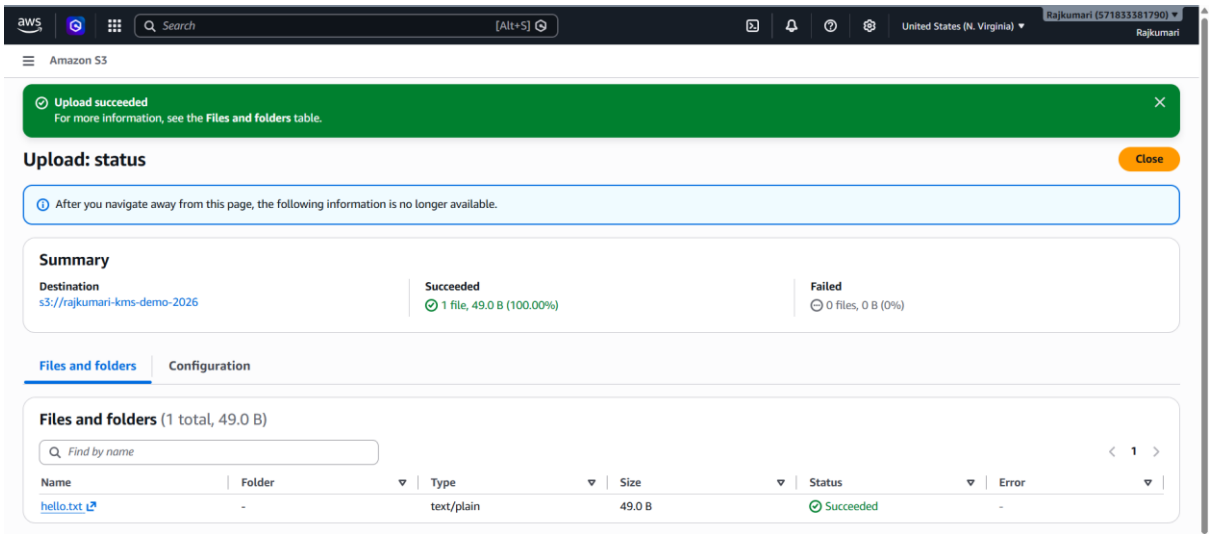
1. Review the configuration.
2. Click **Create bucket**.



Step 5: Upload a Sample File

1. Open the newly created bucket.

2. Click **Upload**.
3. Select a sample file (for example, sample.txt).
4. Click **Upload**.

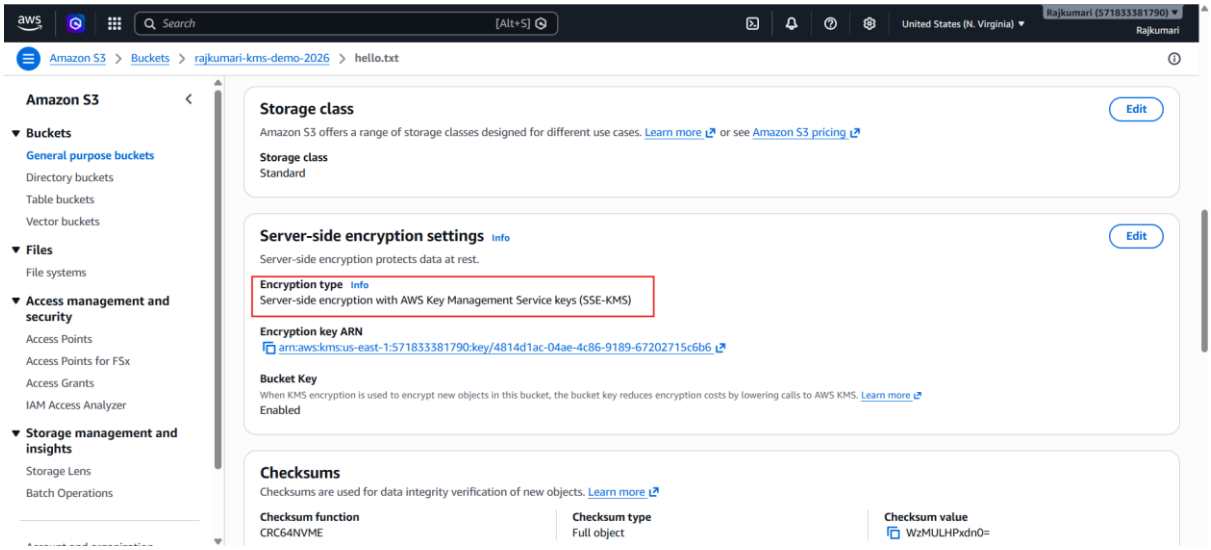


Step 6: Verify the Uploaded Object

1. Open the uploaded object.
2. Go to the **Properties** tab.
3. Scroll to **Server-side encryption settings**.

Verify that:

- Encryption type is **AWS Key Management Service (AWS KMS)**.
- The Customer Managed Key alias (for example, **Project-KMS-Key**) is displayed.

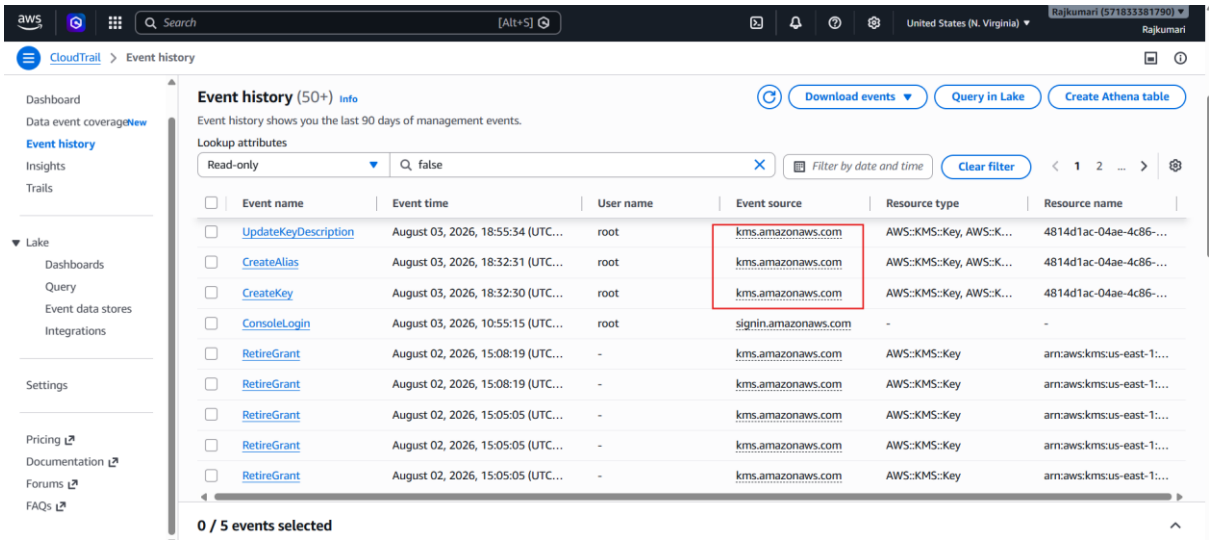


Step 8: Verify KMS Usage in CloudTrail

1. Open **CloudTrail**.
2. Select **Event history**.
3. Filter by **Event source**:

kms.amazonaws.com

4. Look for events such as:
    - GenerateDataKey
    - Encrypt
    - Decrypt (if applicable)



AWS Lambda

AWS Lambda: AWS Lambda is a serverless compute service that allows developers to run code without provisioning or managing servers. Lambda automatically scales based on the number of requests and charges only for the compute time used. It integrates with services such as Amazon S3, API Gateway, CloudWatch, DynamoDB, and SNS.

Objective

- Understand AWS Lambda concepts.
- Create a Lambda function using Python.
- Execute the function with a test event.
- Monitor execution logs using CloudWatch.
- Configure environment variables.
- Understand Lambda execution roles.

Architecture

User

|

| Test Event

▼

AWS Lambda Function

|

| Executes Code

▼

Amazon CloudWatch Logs

Prerequisites

- AWS Account
- IAM user with Lambda permissions
- Region: **us-east-1**
- Internet connection

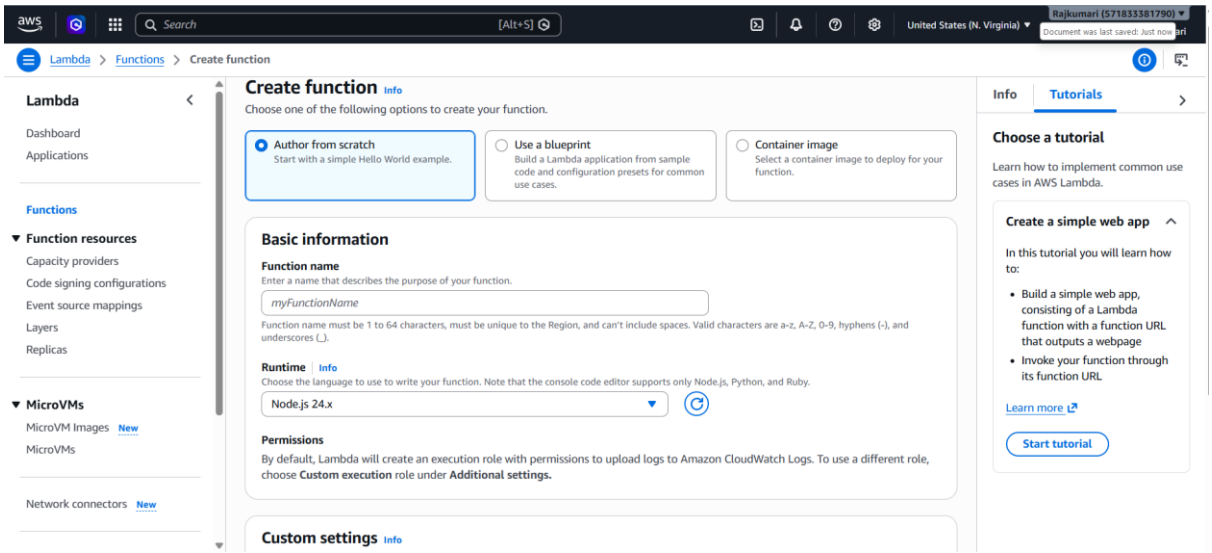
Task 1: Create a Lambda Function

Step 1: Open AWS Lambda

1. Sign in to the AWS Management Console.
2. Search for **Lambda**.
3. Open the **AWS Lambda** service.

Step 2: Create a Function

1. Click **Create function**.
2. Choose **Author from scratch**.

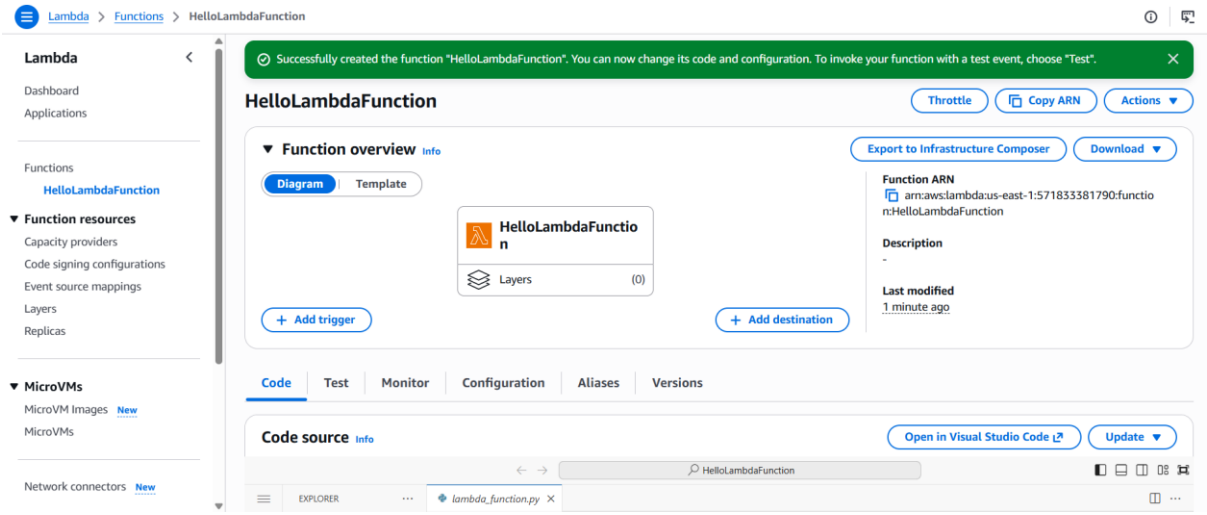


Step 3: Configure the Function

Enter the following details:

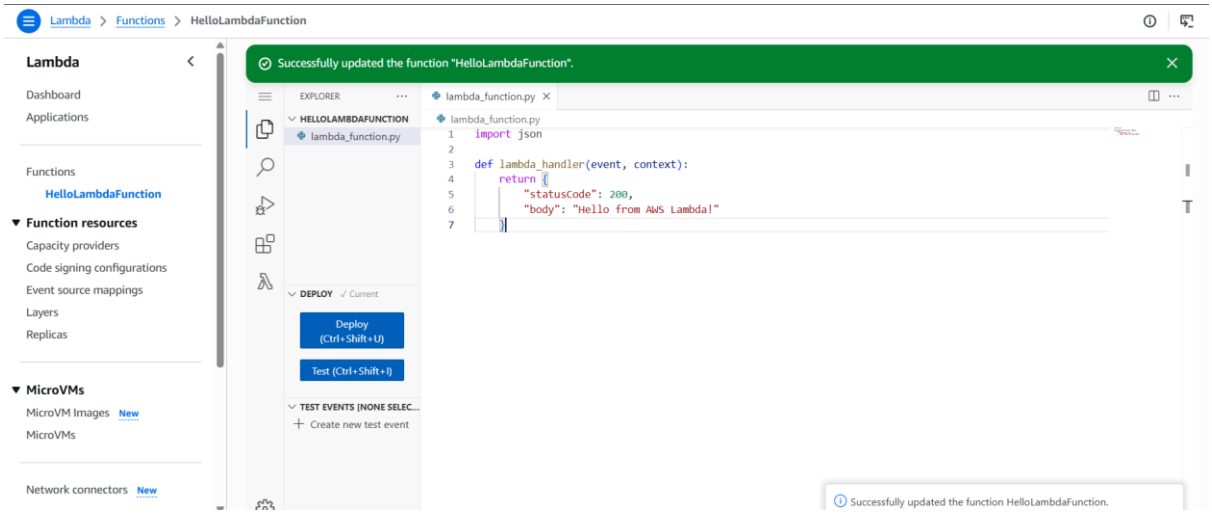
Setting	Value
Function name	HelloLambdaFunction
Runtime	Python 3.13 (or latest available)
Architecture	x86_64
Permissions	Create a new role with basic Lambda permissions

Click **Create function**.



Write the Lambda Code

Click **Deploy**.



Test the Function

Step 1

Click **Test**.

Step 2

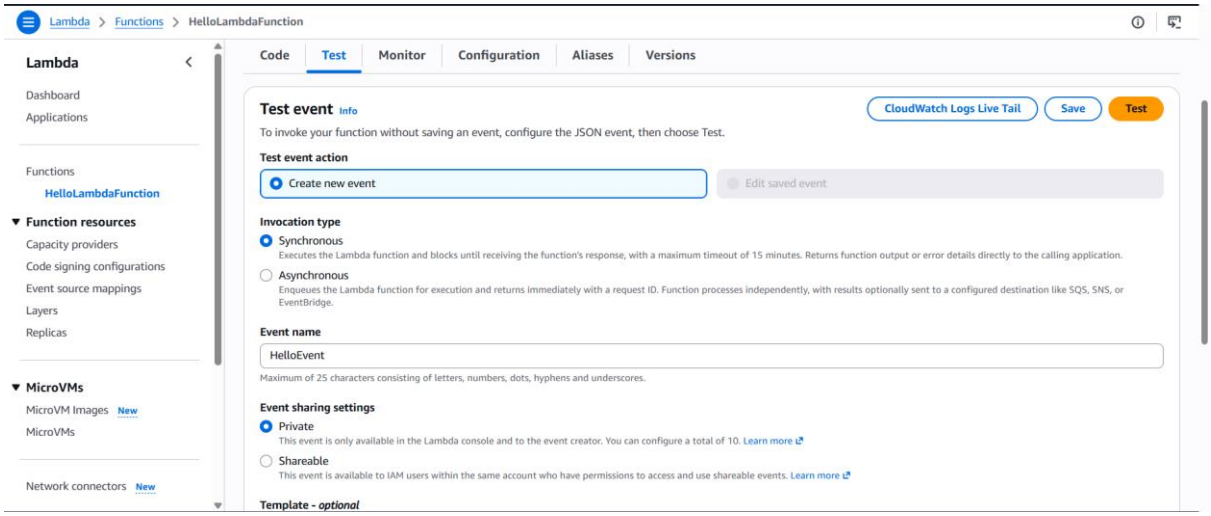
Create a new test event.

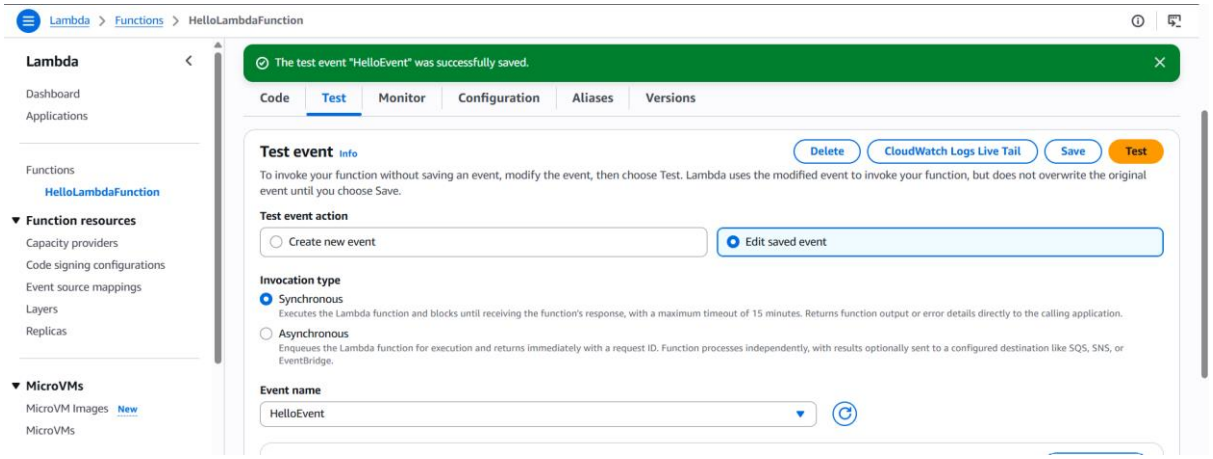
Event name:

HelloEvent

Use the default JSON: {}

Click **Save**.

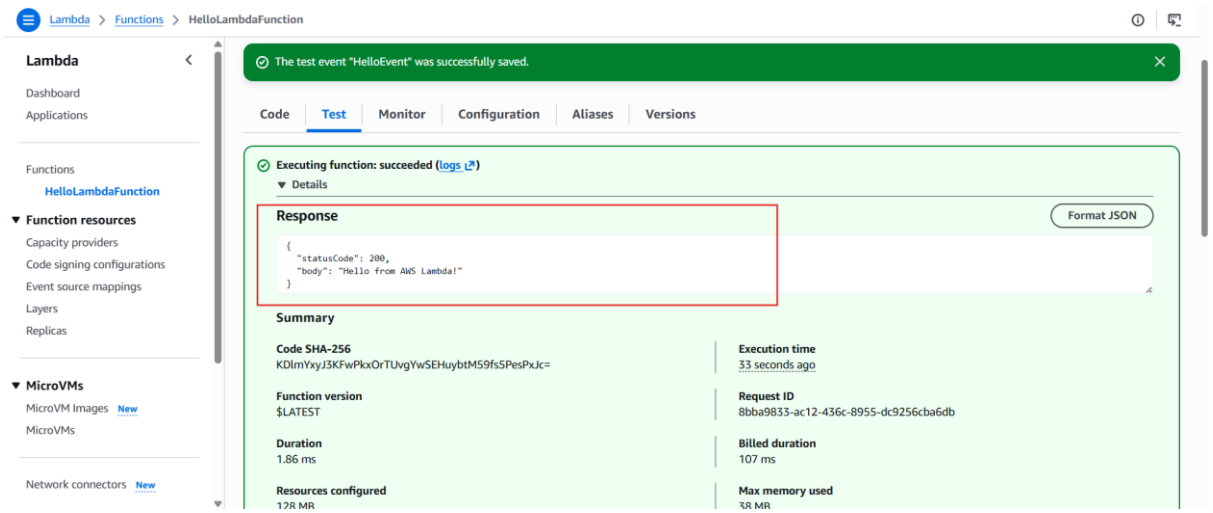


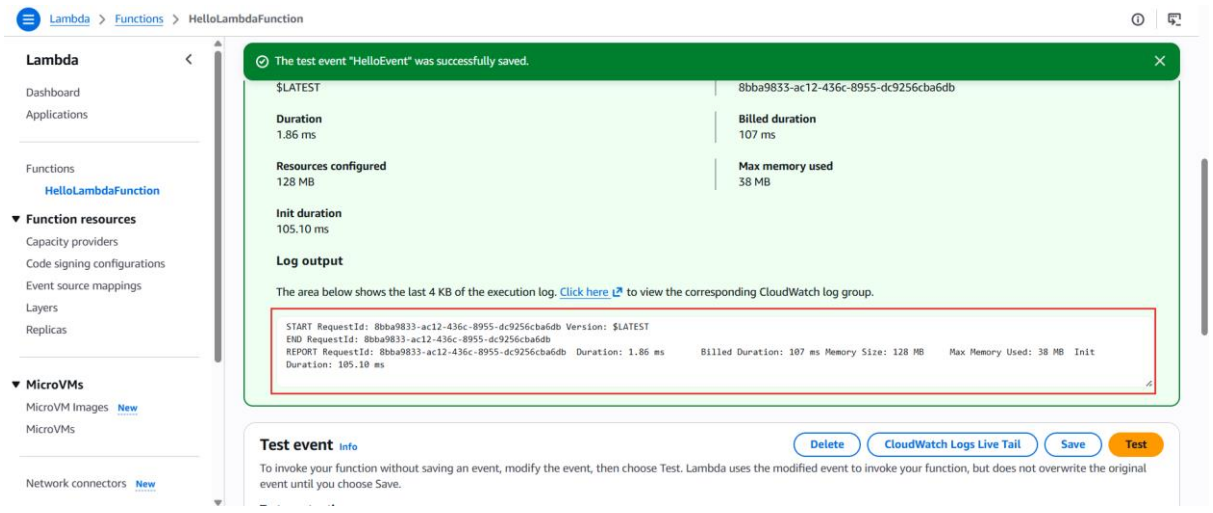


Step 3

Click **Test** again.

Expected output:



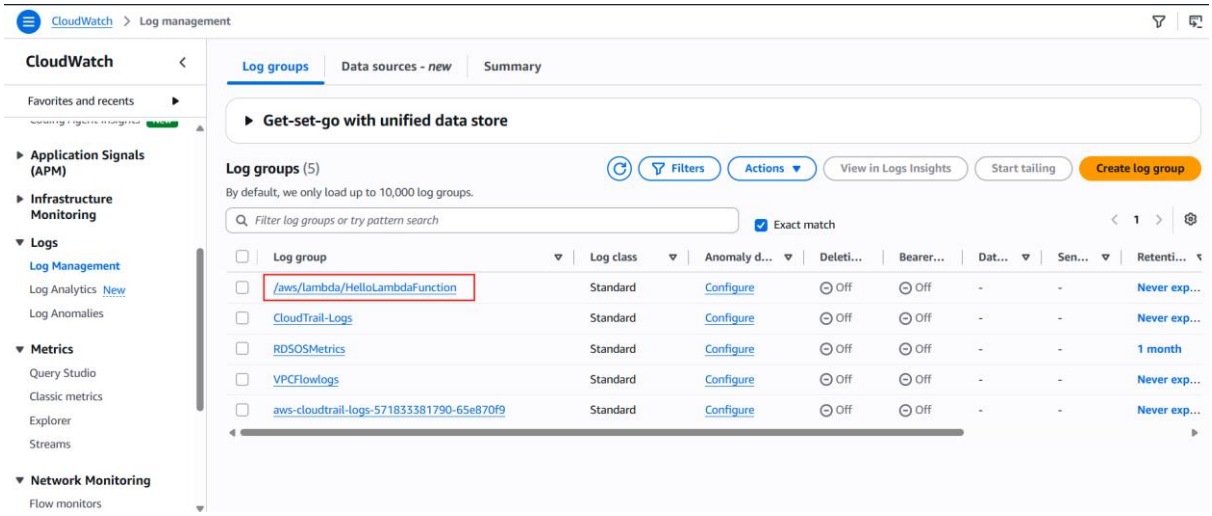


View CloudWatch Logs

Step 1: After testing, select the **Monitor** tab.

Step 2

Click **View CloudWatch Logs**.



Step 3

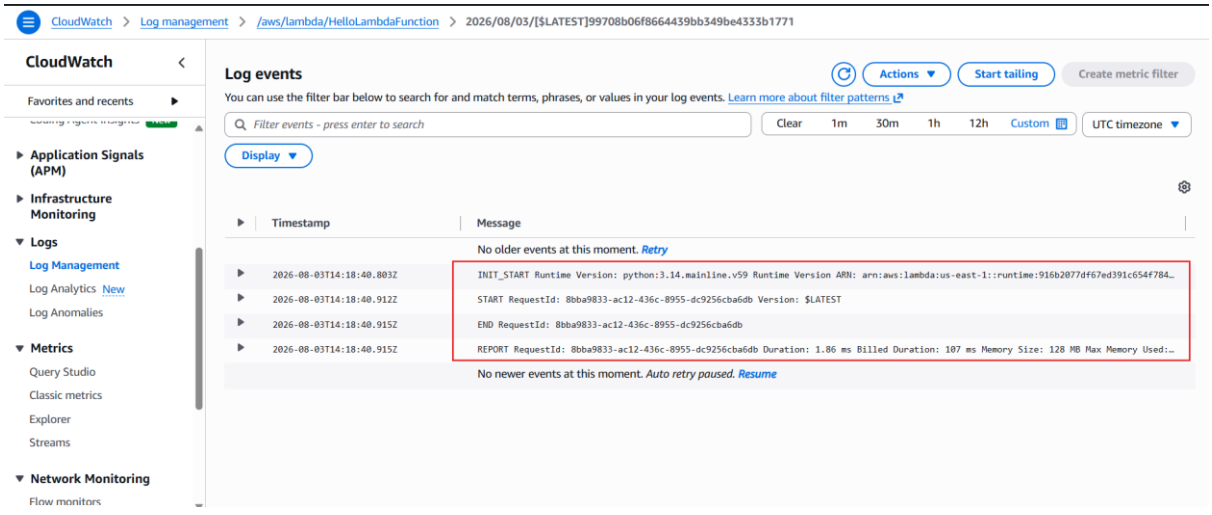
Open the latest log stream.

You should see entries similar to:

START RequestId: ...

END RequestId: ...

REPORT RequestId: ...



Configure Environment Variables

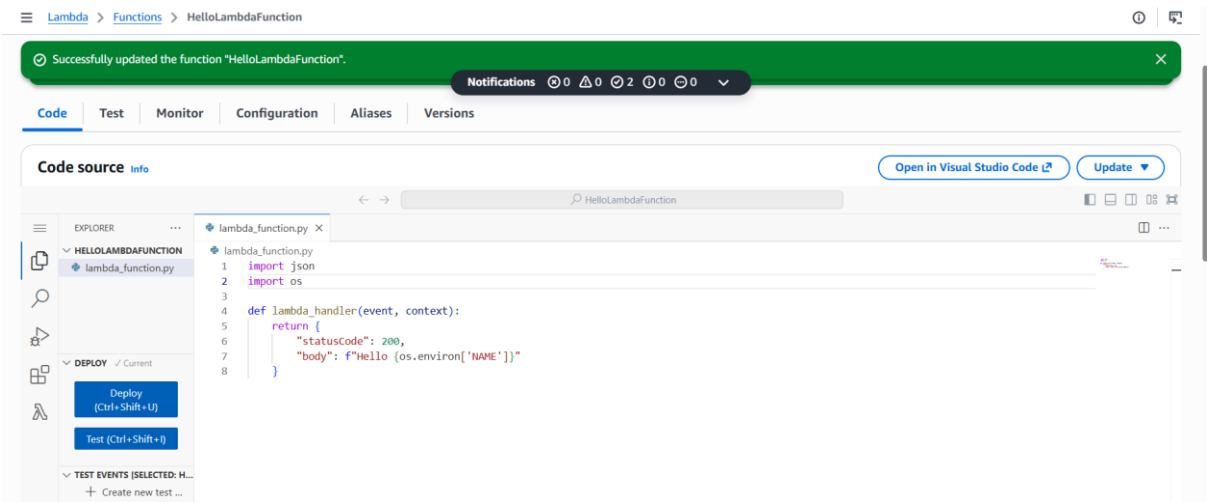
Open the **Configuration** tab.

Select **Environment variables**.

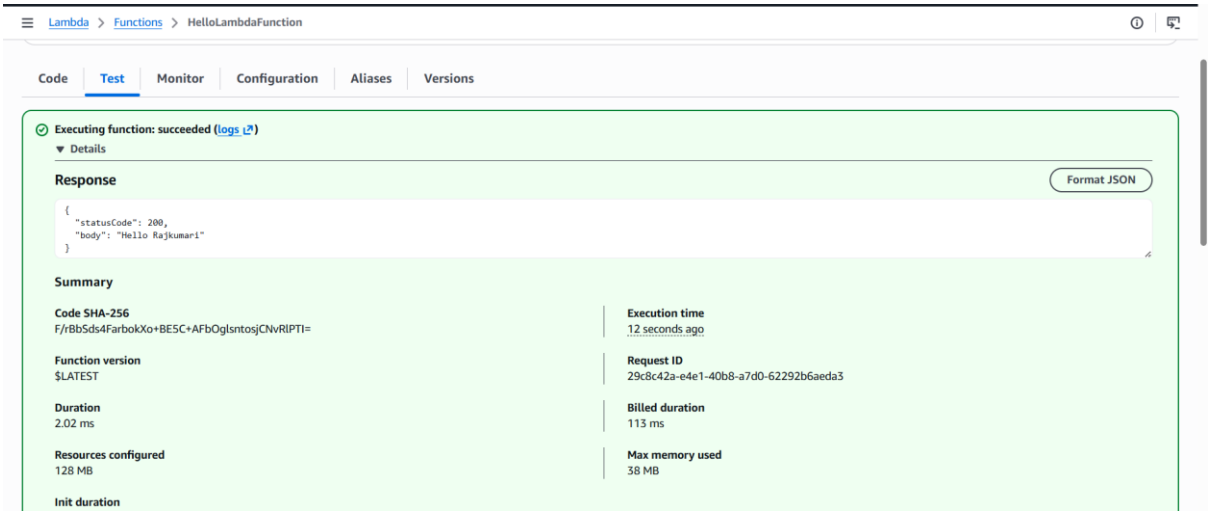
Click **Edit**.

Add a variable:

Key	**Value**
NAME	Rajkumari



Click **Deploy**.



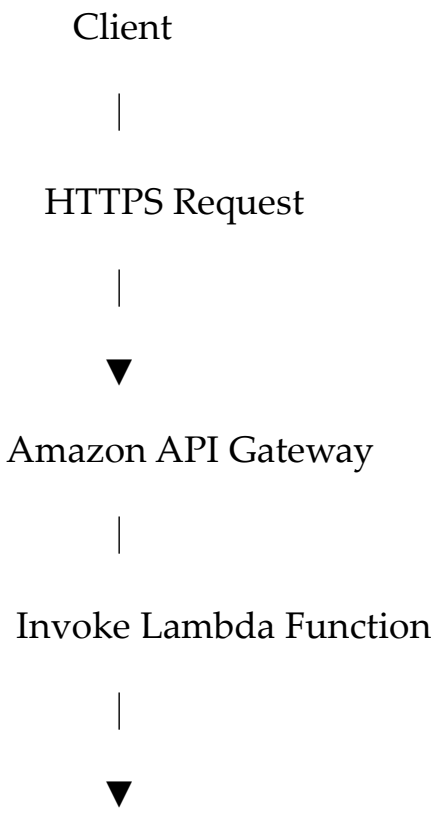
AWS API Gateway

Amazon API Gateway is a fully managed AWS service that enables developers to create, publish, secure, monitor, and maintain APIs. It acts as the front-end interface for backend services such as AWS Lambda, EC2, or other HTTP endpoints.

Objectives

- Understand API Gateway concepts.
- Create a REST API.
- Integrate API Gateway with AWS Lambda.
- Deploy the API.
- Invoke the API from a browser.
- Monitor API execution.

Architecture



AWS Lambda

|

▼

Amazon CloudWatch Logs

Prerequisites

- AWS Account
- Existing Lambda function (HelloLambdaFunction)
- IAM permissions for API Gateway and Lambda
- Region: us-east-1

Create an API

Step 1: Login to the AWS Console.

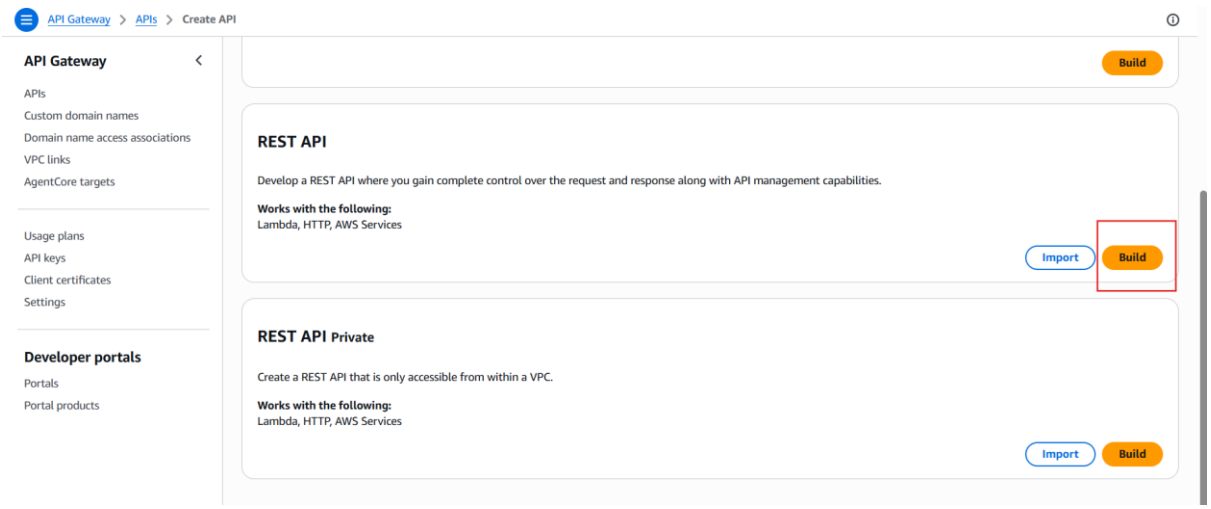
Search for **API Gateway**.

Open the service.

Click **Create API**.

Step 2: Under **REST API**, click **Build**.

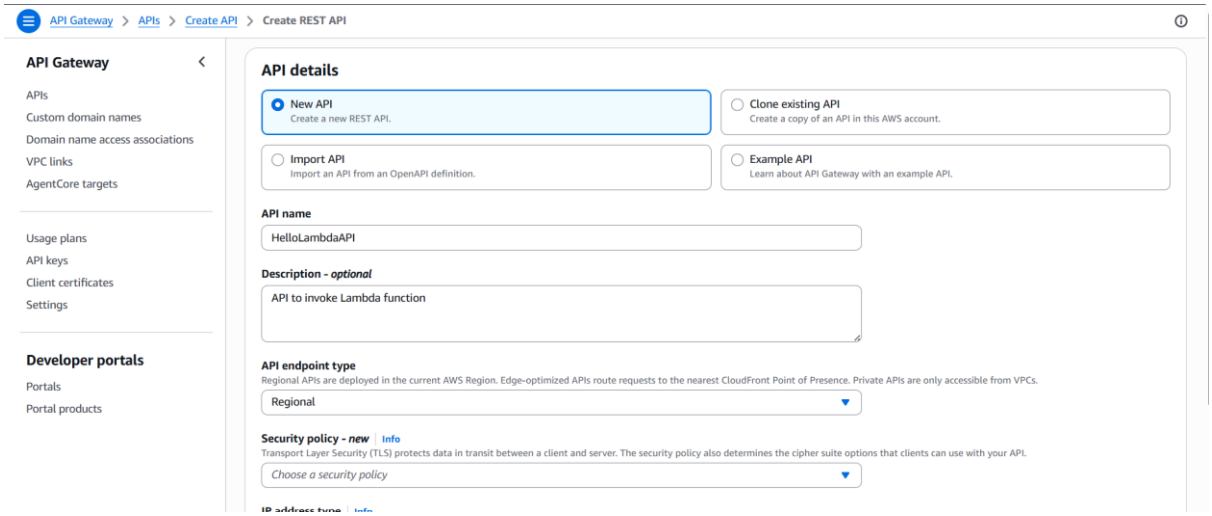
Choose **REST API**, not HTTP API, because it demonstrates more API Gateway features.

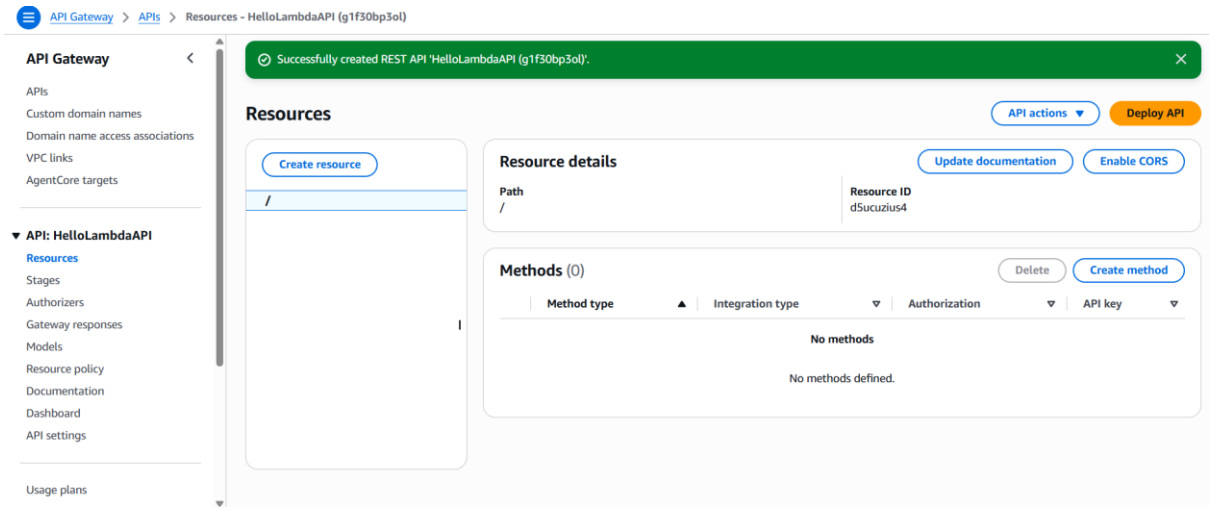


Step 3: Configure the API.

Setting	Value
API name	HelloLambdaAPI
Description	API to invoke Lambda function
Endpoint Type	Regional

Click **Create API**.

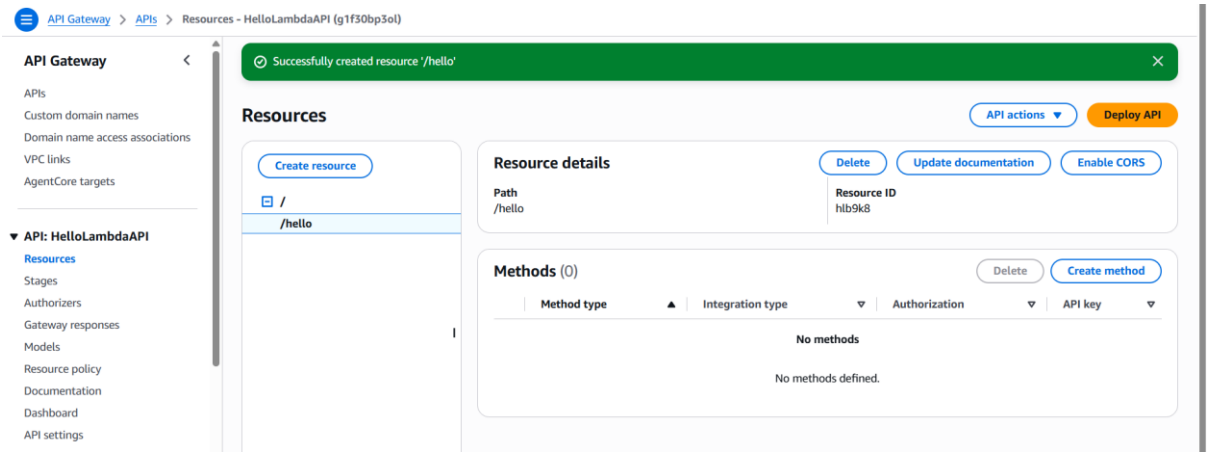




Create a Resource

Step 4: In the left navigation pane, select **Resources**.

Click **Create Resource**.

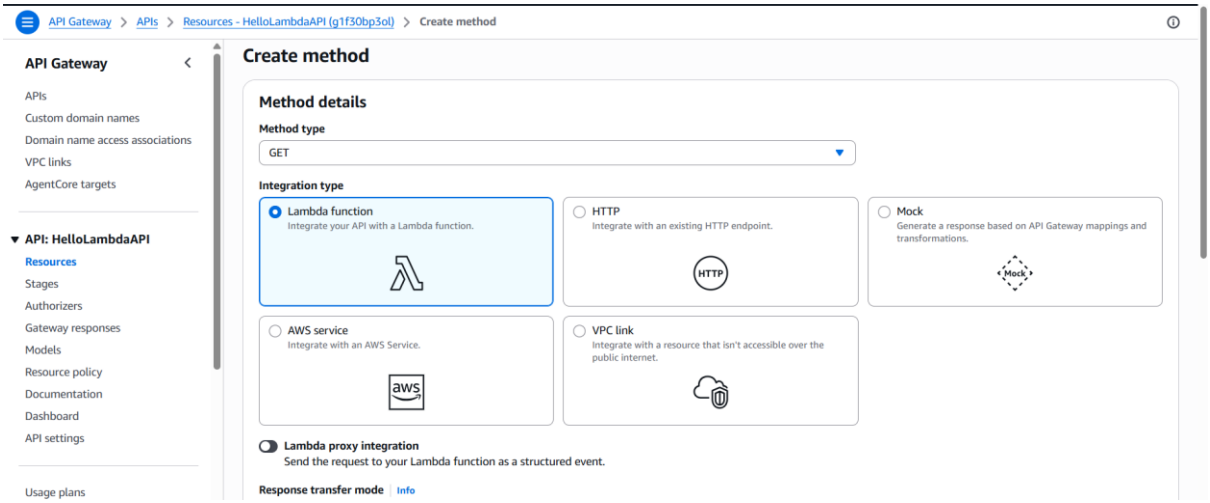


Create GET Method

Step 5: Select the /hello resource.

Click **Create Method** (or **Create** → **Method**, depending on the console).

Choose: GET



Step 6: Configure integration.

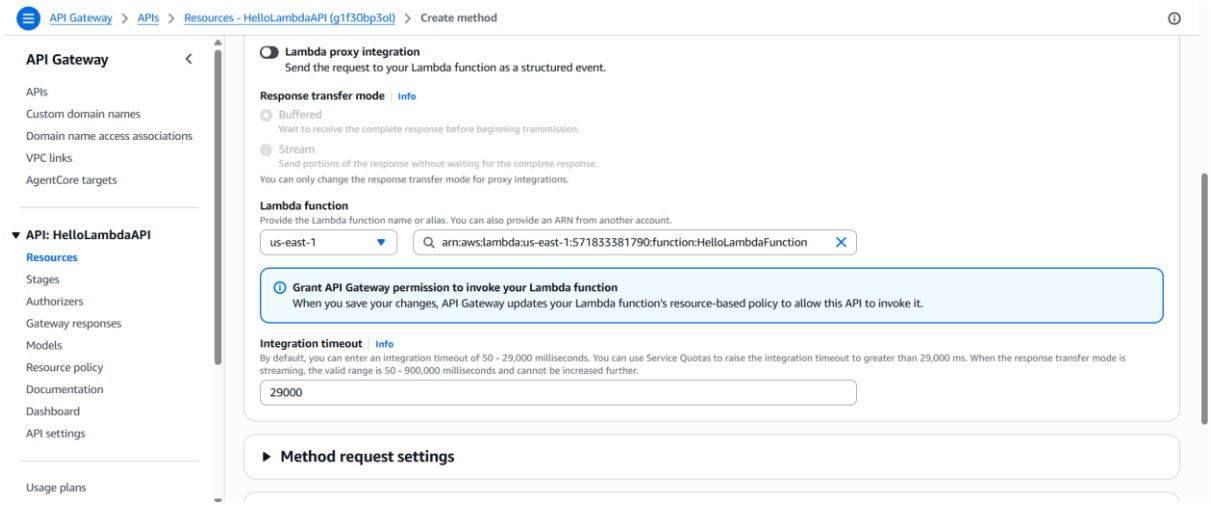
Setting	Value
Integration Type	Lambda Function
Lambda Proxy Integration	Enabled

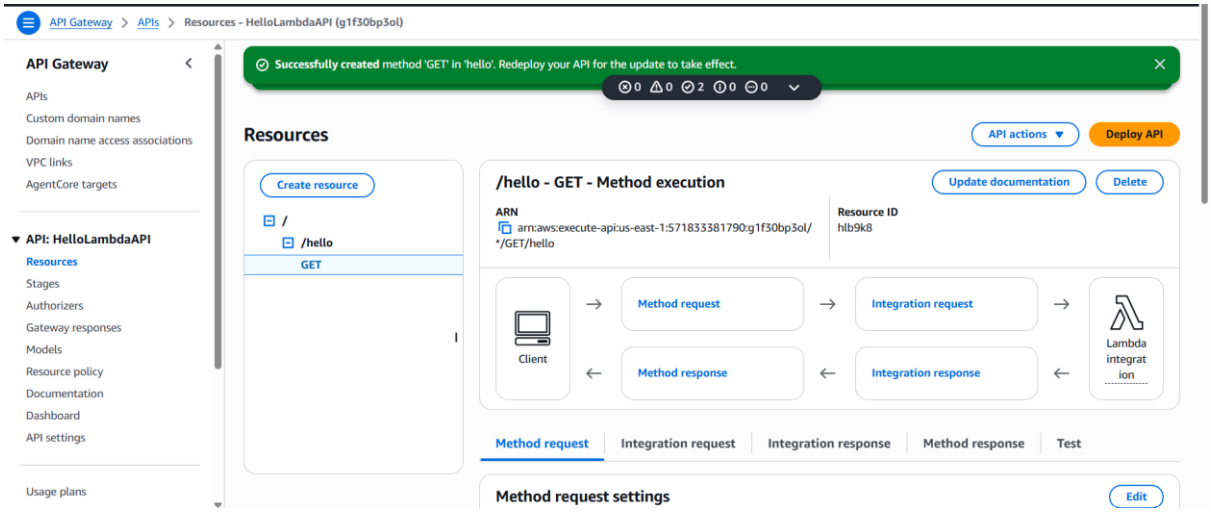
Setting	Value
Region	us-east-1
Lambda Function	HelloLambdaFunction

Click **Create Method** (or **Save**).

When prompted:Add Permission to Lambda?

Choose: OK





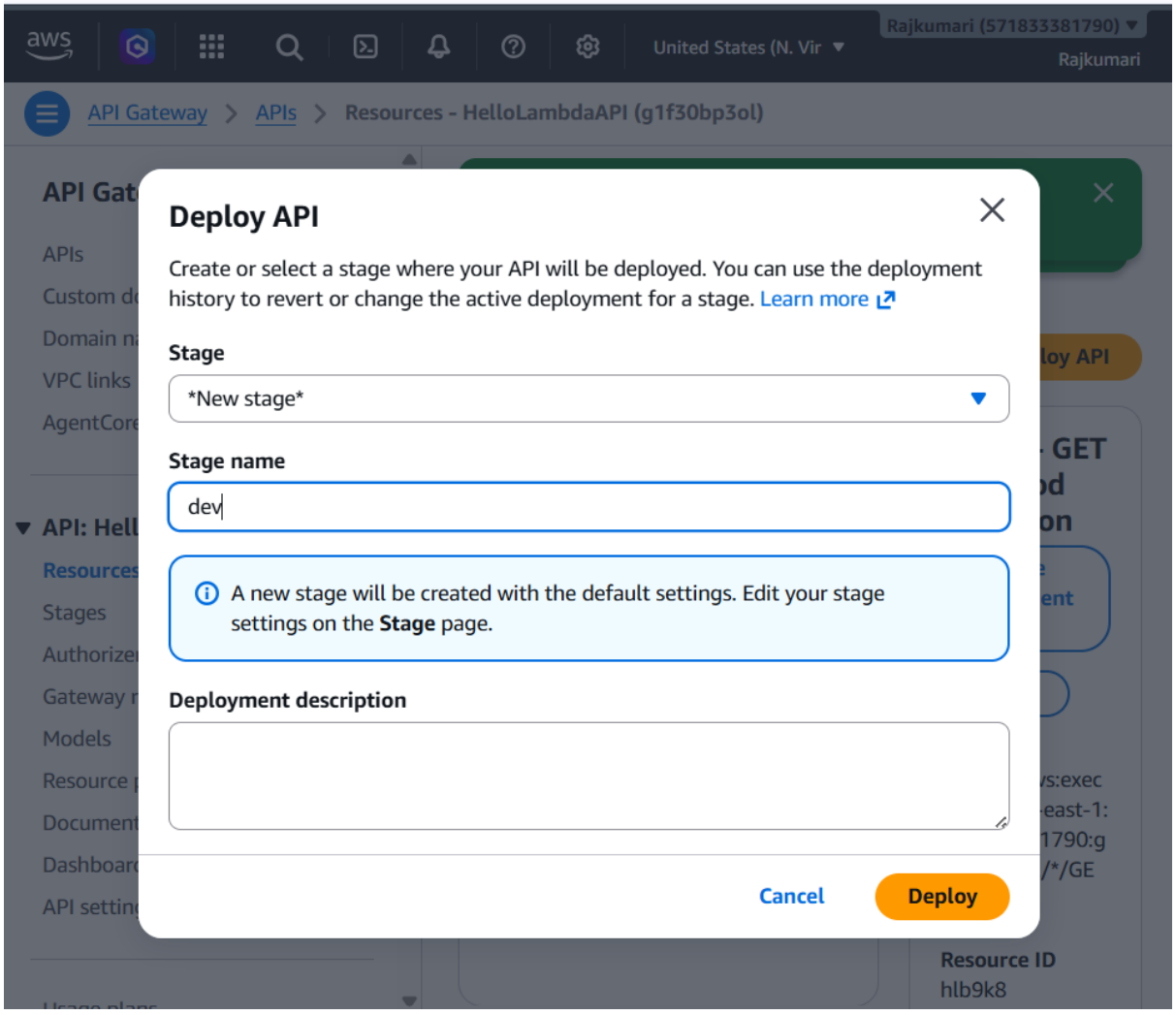
Deploy API

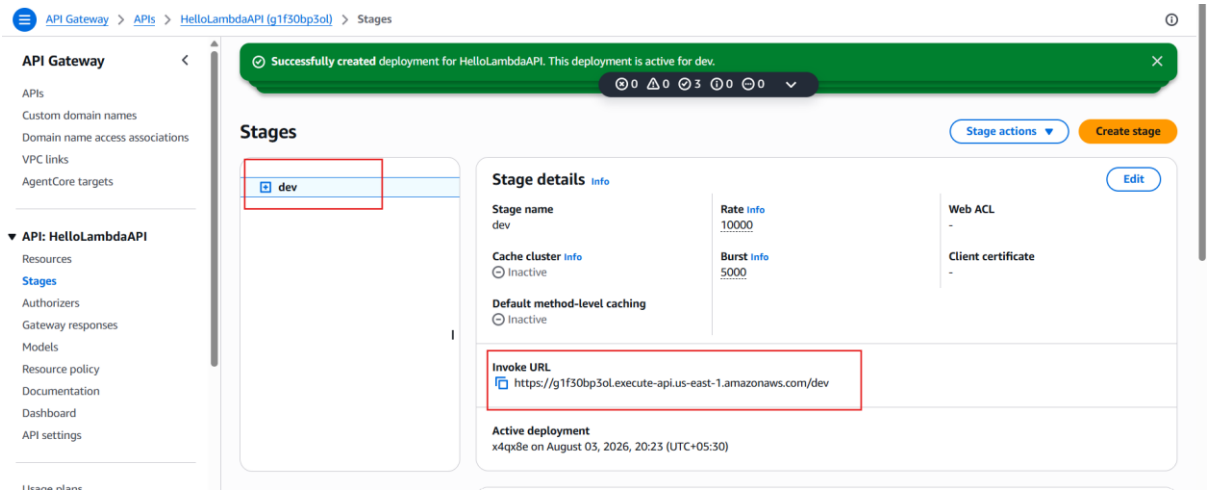
Step 7: Click **Deploy API**.

Create a stage.

Setting	Value
Deployment Stage	New Stage
Stage Name	dev

Click Deploy.





Step 8: After deployment, API Gateway displays an **Invoke URL**.

Example: https://g1f30bp3ol.execute-api.us-east-1.amazonaws.com/dev/hello

Invoke the API

Append the resource path: https://g1f30bp3ol.execute-api.us-east-1.amazonaws.com/dev/hello

Open the URL in a browser.